OS ORANGE PROBLEM-REPORT

NAME:SANJANA V SHASTRY

SRN: PES1UG24AM916

SECTION: G

OPERATING SYSTEMS

ABSTRACT

This project implements a simplified version control system (PES-VCS) inspired by Git. It supports file tracking, staging, commit creation, and history traversal using contentaddressable storage. The system uses SHA-256 hashing to ensure data integrity and efficient storage. The project demonstrates core operating system concepts such as file management, indexing, and persistent storage.

AIM

To design and implement a simplified version control system (PES-VCS) that supports file tracking, staging, commit creation, and history traversal using content-addressable storage.

THEORY

Version Control Systems A Version Control System (VCS) is used to track changes in files over time, allowing users to revert to previous versions and maintain history

Content-Addressable Storage

In this system, files are stored based on their SHA-256 hash. Identical content results in the same hash, ensuring deduplication and integrity.

Object Types

- Blob: Stores file content
- Tree: Represents directory structure
- Commit: Stores metadata and links to tree

Index (Staging Area) : The index temporarily stores files before committing, enabling controlled snapshot creation.

3. IMPLEMENTATION

Phase 1: Object Storage

Implemented functions:

- object_write()
- object_read()

Features:

- SHA-256 hashing
- Directory sharding
- Integrity verification

Screenshot-1A- test_objects output

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot1B -object storage

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$
```

Phase 2: Tree Objects Implemented:

- tree_from_index()

Features:

- Converts index into hierarchical tree
- Supports directory structure representation

Screenshot 2A - test_tree output

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B- xxd output

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aa
f2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Phase 3: Index (Staging Area) Implemented:

- index_load()
- index_save()
- index_add()

Features:

- Tracks staged files
- Stores metadata (mode, size, timestamp)

Screenshot 3A- pes status

```
Initialized empty PES repository in .pes/
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ echo "hello" > file1.txt
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ echo "world > file2.txt
> ./pes add file1.txt file2.txt
> ./pes status
> ^C
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes add file1.txt file2.txt
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  tree.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  README.md
  untracked:  commit.c
  untracked:  object.c
  untracked:  file.txt
  untracked:  commit.h
```

Screenshot3B- index file

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776361605 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776357303 6 file2.txt
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$
```

Phase 4: Commit System Implemented:

• commit_create()

Features:

• Creates commit objects

• Maintains history using parent links

• Updates HEAD reference

Screenshot 4A- pes log

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes commit -m "Add world"
Committed: a74402f4bdd1... Add world
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ echo "bye" > bye.txt
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes add bye,txt
error: failed to add 'bye,txt'
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes add bye.txt
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 9ca12f055686... Add farewell
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ ./pes log
commit 9ca12f0556865b4f6152606d7dfcdc866275a81bd8c3194cc2c452937549d1a3
Author: Sanjana <PES1UG24AM916>
Date:   1776362279

    Add farewell
d far

commit a74402f4bdd1af0dbd5c58214f12460554a6f20e5844f40c66b68d53b3808c1a
Author: Sanjana <PES1UG24AM916>
Date:   1776362249

    Add world
ll
d far

commit 95c181749b9747deff99a45ca00ee5143d52901e94028197d980d1b502043f0a
Author: Sanjana <PES1UG24AM916>
Date:   1776362224

    Initial commit
```

Screenshot 4B- object growth

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/5c/4e1e2f5a561dd416ca159203c3259111f64259b7341e2754484f9b8209b0ae
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/95/c181749b9747deff99a45ca00ee5143d52901e94028197d980d1b502043f0a
.pes/objects/9c/a12f0556865b4f6152606d7dfcdc866275a81bd8c3194cc2c452937549d1a3
.pes/objects/a7/4402f4bdd1af0dbd5c58214f12460554a6f20e5844f40c66b6
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a
.pes/refs/heads/main
```

Screenshot 4C- HEAD

```
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ cat .pes/refs/heads/main
9ca12f0556865b4f6152606d7dfcdc866275a81bd8c3194cc2c452937549d1a3
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ cat.pes/HEAD
bash: cat.pes/HEAD: No such file or directory
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
vboxuser@linuux:~/sanjana_shastry/PES1UG24AM916-pes-vcs$
```

5. ANALYSIS QUESTIONS

Q5.1: Implementation of pes checkout <branch>

A branch in the PES-VCS system is represented as a file in .pes/refs/heads/ that stores the hash of the latest commit. Implementing pes checkout <branch> requires updating both repository metadata and the working directory.

Steps involved:

1. **Branch Validation**
   - Check whether the file .pes/refs/heads/<branch> exists.
   - If it does not exist, return an error indicating an invalid branch.
2. **Read Target Commit**
   - Read the commit hash stored in the branch file.
   - This commit represents the snapshot to which the working directory must be updated.
3. **Update HEAD**
   - Modify the .pes/HEAD file to point to the selected

branch: ref: refs/heads/<branch>

4. **Reconstruct Working Directory**
    - Read the commit object corresponding to the hash.
    - Extract the associated tree object.
    - Traverse the tree recursively:
        - For each blob, read content from the object store and write it to the working directory.
        - For subdirectories, recursively reconstruct structure.
    - Remove any files present in the working directory but not in the target tree.
5. **Update Index**
    - Replace the current .pes/index contents with entries derived from the target tree.

Complexity of the Operation

This operation is complex due to:

- The need to safely overwrite or delete existing files.
- Recursive traversal of tree objects.
- Handling nested directories.
- Ensuring that user data is not lost due to uncommitted changes.

Thus, checkout is not merely a pointer update; it involves complete synchronization between the repository state and the working directory.

Q5.2: Detecting a Dirty Working Directory

A dirty working directory refers to the presence of uncommitted changes in tracked files. Switching branches in such a state may lead to loss of changes if files are overwritten.

Detection Method

1. **Compare Working Directory with Index**
    - For each file tracked in the index:
        - Compare file size and modification time with stored values.
        - If differences are found, the file is considered modified.
2. **Compare Index with Target Branch Tree**
    - Load the tree corresponding to the target branch.
    - Compare file hashes in the index with those in the target tree.

3. **Conflict Detection**
    - A conflict occurs if:
        - A file has been modified in the working directory, and
        - The same file differs in the target branch.
4. **Abort Checkout**
    - If a conflict is detected, the operation should be aborted with an appropriate error message.

Justification

The index represents the last staged state, while the object store represents committed snapshots. By comparing these, it is possible to detect inconsistencies and prevent

unsafe operations.

Q5.3: Detached HEAD State

A detached HEAD occurs when the .pes/HEAD file contains a direct commit hash instead of a reference to a branch.

Behavior

- New commits can still be created.
- However, these commits are not associated with any branch.
- They form an independent chain of commits.

Implications

- These commits are not reachable through any branch.
- They may be lost if not referenced explicitly.
- They are not visible in standard branch-based history traversal.

Recovery Mechanisms

1. **Creating a New Branch**
    - A new branch can be created pointing to the detached commit.
2. **Manual Reference Update**
    - The commit hash can be written into a new reference file.
3. **Using Commit Hash Directly**
    - The commit can still be accessed using its hash.

Detached HEAD is useful for temporary exploration or debugging, but it requires careful handling to avoid losing work.

---

6. GARBAGE COLLECTION AND SPACE RECLAMATION

Q6.1: Algorithm for Removing Unreachable Objects

Over time, the object store accumulates objects that are no longer referenced by any commit or branch. These objects must be identified and removed to reclaim space.

Algorithm

1. **Identify Root References**
   - Read all branch references from .pes/refs/heads/.
   - Each reference provides a starting commit.
2. **Traverse Commit Graph**
   - Perform a Depth-First Search (DFS) or Breadth-First Search (BFS):
     - Visit each commit.
     - Add its parent commit (if any).
     - Add its associated tree.
2. **Traverse Tree Objects**
   - For each tree:
     - Add all blobs and subtrees referenced by it.
3. **Mark Reachable Objects**
   - Maintain a hash set to store all reachable object hashes.
4. **Sweep Phase**
   - Traverse all objects in .pes/objects/.
   - Delete any object not present in the reachable set.

Data Structure

A hash set is used to store reachable object identifiers, ensuring constant-time lookup during the sweep phase.

Complexity

For a repository with 100,000 commits and multiple branches:

- All commits may need to be visited.
- Trees and blobs increase the total number of objects.
- However, the use of efficient data structures ensures acceptable performance.

Q6.2: Garbage Collection Race Condition

Running garbage collection concurrently with commit operations can lead to inconsistencies.

Race Condition Scenario

1. Garbage collection begins scanning objects.
2. A new commit is being created simultaneously.
3. The commit references a tree or blob.
4. The garbage collector has not yet marked this object as reachable.
5. The object is incorrectly deleted.
6. The commit becomes invalid due to missing references.

Consequences

- Loss of repository integrity.
- Corrupted commit history.
- Inability to retrieve data.

Preventive Measures

1. **Locking Mechanisms**
    - Prevent simultaneous execution of commit and garbage collection.
2. **Reference Tracking**
    - Temporarily mark objects being created as protected.
3. **Two-Phase Garbage Collection**
    - First mark all reachable objects.
    - Then delete unreachable objects.
4. **Grace Period**
    - Delay deletion of newly created objects to avoid premature removal.

Conclusion

To ensure consistency, garbage collection must be carefully synchronized with ongoing repository operations. Proper design prevents accidental data loss and maintains

system integrity.